PHOBIA RELIEF THERAPY IN VR

FYP Viva: Technical Panel Defense Guide & Q&A; Simulation

This preparation document simulates a final-year project (FYP) thesis panel defense. It features four simulated examiner personalities, each looking at the project through a different lens (System Architecture, Graphics/VR, Databases/Security, and UX/Validation). Preparing these answers ensures your team defends the project comprehensively.

Simulated Examiner Panel Profiles

Examiner Persona	Domain Focus	Core Technical Interest Areas
Dr. Aris	Software Architecture & Systems Design	Design patterns (Observer, Style Guide), dynamic code generation, coroutines/asynchronous flows, system modularity, clean code, SQLite schema interfaces.
Prof. Sarah	VR/AR Interactions & Computer Graphics	XR Interaction Toolkit (XRI) event layers, coordinate spaces (Screen vs. World space), VR tracking offsets (Double-height fix), material panoramic shaders, 3D physics raycasting.
Dr. Fatima	Database, Security & Cloud Engineering	Supabase cloud schema, relational modeling (PostgreSQL), data synchronization, offline SQLite fallback layer, secure Client APIs, and access token caching.
Dr. John	UX/UI, Human-Computer Interaction & Validation	Comfort physics, virtual keyboard overlays, input validation (fields/passwords), BLE sensor integration (Polar H10), physiological heart rate (BPM) mock simulation, baseline calibration.

Important Notice to FYP Students: All answers compiled in this guide directly map to actual production scripts inside your project repository. Ensure your partners are familiar with specific files (e.g., *VRRigBuilder.cs*, *VRMenuDragger.cs*, *SensorManager.cs*) so you can quote lines of code and physical logic rules during questioning.

Q1: Why did you build the VR camera rig dynamically at runtime using code (VRRigBuilder.cs) rather than keeping predefined prefab rigs in each scene?

Asked by: Dr. Aris (Software Architecture & Systems Design)

Answer: By dynamically constructing the VR camera rig at runtime, we avoid scene replication and synchronization issues across our 11 scenes. If a prefab rig was static in each scene, any change to tracking parameters, input configurations, comfort wrist angles, or raycasters would require manual editing and testing of all 11 scenes. The dynamic builder establishes a single source of truth, reducing human error, scene asset file sizes, and development overhead.

Q2: What is the 'Double-Height Camera Fix' inside VRRigBuilder.cs, and how does your script prevent height calculation stacking when deploying to physical VR devices?

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: When a standalone VR headset (like the Meta Quest) initializes in 'Floor' tracking mode, the device offset already reflects the user's physical standing height. If our root XROrigin GameObject in the scene is preset to a default camera level (e.g., 1.7m), these heights accumulate, making the camera float at 3.4m in space. We resolve this by ensuring the camera origin root coordinates are set strictly to floor level (Y = 0f) on initialization, allowing hardware-tracked offsets to determine height correctly.

Q3: Since standard Unity canvases render in Screen Space (which doesn't display in VR), how does your application dynamically convert them, adjust their spatial scale, and bind input pointer raycasters?

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: Overlay canvases are incompatible with VR. We programmatically convert them to WorldSpace canvases, downscale their Transform scales to 0.001f (so 1920 pixels equals 1.92m in game space), and position them at a comfortable reading distance of 1.5m from the camera. We swap the default GraphicRaycaster with the XR Interaction Toolkit's TrackedDeviceGraphicRaycaster, allowing the controllers' laser-pointer rays to trigger UI elements.

Q4: How is the menu grabbing and dragging mechanism (VRMenuDragger.cs) implemented, and how do you calculate the controller grip rotation and thumbstick push/pull distances at runtime?

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: We dynamically add a BoxCollider matching the Canvas dimensions at runtime. When the controller ray targets the collider and the user presses the Grip button, the menu anchors. During drag, we position the canvas relative to the controller's forward ray. The user can also move the Thumbstick Y-axis to dynamically push/pull the canvas distance (clamped between 0.5m and 3.0m), and the X-axis to adjust yaw, providing spatial convenience.

Q5: Why did you choose to build a custom, code-driven virtual keyboard (VRKeyboard.cs) instead of using the headset's native OS keyboard overlay, and how did you implement password masking for it?

Asked by: Dr. John (UX/UI, Human-Computer Interaction & Clinical Validation)

Answer: Headset operating system overlay keyboards (such as the default Android keyboard on Oculus Quest) run outside of Unity's rendering pipeline. This often causes stereoscopic depth glitches, alignment errors, or graphics engine crashes. Creating a custom virtual keyboard built entirely from Unity UI buttons in WorldSpace ensures unified rendering, allows custom key positioning, supports drag-handling, and integrates custom password masking securely.

Q6: Why did you choose Supabase over Firebase as your backend database provider?

Asked by: Dr. Fatima (Database, Security & Cloud Engineering)

Answer: Supabase is built on PostgreSQL (relational SQL), which enforces structural integrity and foreign key relationships (e.g., linking logs in a sessions table to specific accounts in a profiles table). Firebase is a NoSQL JSON database, which makes complex, nested relational queries difficult to write and verify. Supabase also supports secure PostgreSQL triggers and stored procedures directly on the database server.

Q7: How does the database system handle temporary internet disconnections or network drops during a therapy session?

Asked by: Dr. Fatima (Database, Security & Cloud Engineering)

Answer: We implemented a hybrid storage system. Local database cache operations are handled by a local SQLite database (PhobiaReliefTherapy.db loaded via the SQLite4Unity3d library). When a session completes, its data is logged to the SQLite database. If the internet is active, it immediately calls the Supabase API to sync. If offline, the session is queued in SQLite, and a background check syncs the cached records once an internet connection is established.

Q8: Explain the design pattern used to construct your medical styling engine (UIThemeAutoApply.cs & MedicalTheme ScriptableObject). How are visual tokens applied?

Asked by: Dr. Aris (Software Architecture & Systems Design)

Answer: We use the Observer/Style Guide pattern coupled with Unity ScriptableObjects. A centralized MedicalTheme ScriptableObject holds style tokens (e.g. Navy background, clinical teal buttons, TextMesh Pro font styles). Monobehaviour components called ThemeableUI are attached to UI widgets to declare their element type (e.g. PrimaryButton, Header). At scene load, UIThemeAutoApply sweeps the active hierarchy and applies these tokens dynamically, enforcing consistent styling project-wide.

Q9: How does the heart rate tracking subsystem function, and how is validation conducted when no physical BLE sensor is connected?

Asked by: Dr. John (UX/UI, Human-Computer Interaction & Clinical Validation)

Answer: The application includes a SensorManager that utilizes a Bluetooth Low Energy (BLE) API to connect to hardware sensors like the Polar H10. When no sensor is found (or when running in the Editor), it activates a Mock Sensor Fallback algorithm. The mock engine generates a baseline heart rate (e.g. 70-80 BPM) with simulated physical noise (+/- 2 BPM), and increases it dynamically by 15-25% under phobia exposure tracks to simulate stress response, allowing testing of validation logic.

Q10: Why did you use Unity Coroutines for baseline calibration (20s) and web requests? What would happen if standard synchronous thread blocking (e.g. Thread.Sleep) were used in a VR application?

Asked by: Dr. Aris (Software Architecture & Systems Design)

Answer: Unity runs on a single-threaded main game loop. Synchronous blocking operations stall this thread, causing frames to drop, which in VR leads to instant latency lag and motion sickness. Unity Coroutines use cooperative multitasking via yield return, allowing network requests (UnityWebRequestAsyncOperation) and the 20s baseline timer to run asynchronously over multiple frames, maintaining a smooth VR refresh rate (e.g., 90Hz).

Q11: Explain how you project 360-degree panoramic images into a 3D space, and how your flat-screen fallback simulates rotation in the Unity Editor.

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: We load a 360-degree equirectangular panoramic texture at runtime, assign it to a custom Material utilizing the Skybox/Panoramic shader, and apply it to RenderSettings.skybox. Headset rotation is updated by the TrackedPoseDriver. In the Editor, a flat 2D Canvas background displays the fallback image, and holding the Right Mouse Button hides it and lets you pan the camera via Mouse X/Y drag to simulate headset look-around.

Q12: How are user credentials and session tokens managed on the client device, and what security practices are implemented?

Asked by: Dr. Fatima (Database, Security & Cloud Engineering)

Answer: When Supabase validates credentials via AuthManager.cs, it returns an access token which is cached locally using secure PlayerPrefs keys. For email-based password recovery, the URL redirect fragment containing the recovery hash is intercepted by docs/index.html and routed to a dedicated password-reset page, where the client calls the Supabase SDK to safely rewrite credentials, preventing token exposure.

Q13: How is the Singleton pattern implemented in your managers (such as AuthManager or DatabaseManager), and how do you prevent duplicate manager instances during scene transitions?

Asked by: Dr. Aris (Software Architecture & Systems Design)

Answer: We use a thread-safe static instance pattern with a lazy initializer or an Awake() checkpoint: public static AuthManager Instance { get; private set; }. To prevent duplicate manager instances when scenes reload, we run a check: if Instance already exists, we destroy the duplicate GameObject (Destroy(gameObject)). Otherwise, we assign the instance and call DontDestroyOnLoad(gameObject) to persist the instance across scene transitions, maintaining a continuous session lifecycle.

Q14: How does the gaze pointer tracking mechanism (VRGazePointer.cs) calculate whether the user is looking at a phobia trigger?

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: VRGazePointer projects a mathematical Ray forward from the Main Camera's center transform (Camera.main.transform.position and Camera.main.transform.forward). It performs a 3D physics raycast using Physics.Raycast(ray, out hit, maxDistance, triggerLayerMask). If the ray hits a collider assigned to a phobia trigger, it registers a 'look-at' state. If the patient turns their head and the ray no longer hits the collider, it logs a 'look-away' state, allowing us to calculate exposure metrics.

Q15: How is patient data security maintained when transmitting session metrics to the database? How are JWT tokens and API requests secured?

Asked by: Dr. Fatima (Database, Security & Cloud Engineering)

Answer: We enforce Row Level Security (RLS) in Supabase. Every HTTP request carries a JSON Web Token (JWT) in the 'Authorization: Bearer ' header, received upon login. Supabase inspects this JWT to authenticate the user and verifies that the user_id column of the row matches the authenticated user ID (auth.uid() = user_id), preventing any unauthorized user or interceptor from viewing, creating, or editing other patients' logs.

Q16: In the exposure tracks (like the progressive darkness therapy), why are the exposure segments broken down into 3 stages of 30 seconds instead of a single continuous exposure, and how does the UI support this?

Asked by: Dr. John (UX/UI, Human-Computer Interaction & Clinical Validation)

Answer: Clinical exposure therapy relies on systematic desensitization and stimulus grading. Moving through distinct stages (Stage 1, Stage 2, Stage 3) with increasing intensity (e.g. progressive levels of darkness) builds tolerance step-by-step. The UI supports this by displaying a dedicated 'Stage X' overlay text alongside the countdown timer, keeping the user oriented, reducing surprise panic, and allowing therapists to see at which stage the patient's heart rate spikes.

Q17: How do you prevent simulator sickness (motion sickness) in your application, especially considering the patient is already under anxiety?

Asked by: Dr. John (UX/UI, Human-Computer Interaction & Clinical Validation)

Answer: We use zero active locomotion. Patients sit or stand in a fixed physical space, and the camera movements match their physical head turns 1:1 via OpenXR. There are no smooth controller thumbstick translations, which are the main cause of sensory conflict (where the visual system registers motion but the vestibular system does not). We also place a persistent flat menu panel at 1.5m distance, serving as a stable visual anchor point.

Q18: How does the controller laser pointer detect and interact with the custom VR virtual keyboard keys?

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: The XR Interaction Toolkit's XRRayInteractor component on the controller GameObject casts a 3D line forward. The virtual keyboard keys are standard Unity UI Button components rendering in WorldSpace, containing box colliders. When the controller ray collides with a key, XRI's XRUIInputModule maps the laser hit coordinates, highlights the button, and on trigger press, registers the click event which fires the keyboard's input handler.

Q19: How is the SQLite database initialized and accessed programmatically on standalone mobile devices (like Android-based VR headsets)?

Asked by: Dr. Fatima (Database, Security & Cloud Engineering)

Answer: We use SQLite4Unity3d which handles multi-platform path generation. The database path is set to Application.persistentDataPath + '/PhobiaReliefTherapy.db'. On Android (Oculus Quest), this resolves to the secure sandboxed storage path '/storage/emulated/0/Android/data//files/'. SQLite is opened using a file connection object, avoiding the need for network access or external database servers.

Q20: How do SensorManager.cs, BaselineManager.cs, and DatabaseManager.cs communicate session data between each other?

Asked by: Dr. Aris (Software Architecture & Systems Design)

Answer: We use a centralized session data context model called UserData.cs and a controller state machine. SensorManager acts as a data provider, writing real-time heart rate values to a public getter. During calibration, BaselineManager samples this getter every second, computes the baseline average, and stores it. In the final scene, this baseline, along with the exposure session statistics, is compiled into a JSON payload and sent to DatabaseManager.cs to trigger the SQLite and Supabase synchronizations.

Q21: Why did you choose 360-degree equirectangular panoramic textures instead of rendering fully modeled 3D models for the phobia exposure scenarios?

Asked by: Prof. Sarah (VR/AR Interactions & Computer Graphics)

Answer: Rendering fully modeled 3D environments with millions of polygons and real-time lighting is computationally expensive and raises the APK size. On standalone mobile HMDs like the Quest, this can drop the frame rate below the target 90Hz, causing lag and motion sickness. Panoramic skyboxes render extremely fast (a single draw call), require minimal memory, and provide photographic clinical realism (such as height or crowd settings), ensuring smooth performance.

Q22: How do you authenticate users securely in Supabase without exposing your database credentials in the compiled Unity binary?

Asked by: Dr. Fatima (Database, Security & Cloud Engineering)

Answer: We access Supabase via its secure HTTP REST Gateway using a public API key (Anon Key) combined with User JWTs. The client binary only contains the public Anon Key, which has restricted permissions. When a user logs in with their email and password, Supabase returns a JSON Web Token (JWT) representing their session. All subsequent data insert/query operations are authorized using this user JWT. Row Level Security (RLS) on PostgreSQL validates that the token has access to the requested table rows, meaning database credentials are never stored or exposed on the client.